

[1 TP3057 PCM 码转线性电压值 — 完整说明](#)

[1 1. TP3057 PCM 编码格式](#)

[1 1.1 基本参数](#)

[2 1.2 8-bit A-law PCM 编码位结构详解](#)

[1 为什么需要 A-law 压缩?](#)

[2 8-bit 结构: 1 + 3 + 4](#)

[1 bit7 — 符号位 \(Sign\)](#)

[2 bit6–bit4 — 段号 \(Segment, 0–7\)](#)

[3 bit3–bit0 — 步进号 \(Step, 0–15\)](#)

[3 段号控制步长的直观图示](#)

[4 解码过程完整示例 \(以 0xAA = +Full Scale 为例\)](#)

[5 代码中 bit 提取的完整流程 \(以求 0xAA 为例\)](#)

[6 编码字段总结](#)

[3 1.3 偶位反转](#)

[4 1.4 手册中的编码对照表 \(p.11\)](#)

[2 2. 关键电压参数 \(手册 p.9 Transmission Characteristics\)](#)

[3 3. 转换流程](#)

[1 3.1 三步法总览](#)

[2 3.2 步骤 1: 去除偶位反转](#)

[3 3.3 步骤 2: G.711 A-law 解码 → 13-bit 线性值](#)

[1 3.3.1 A-law 字节结构](#)

[2 3.3.2 A-law 段落结构](#)

[3 3.3.3 解码公式 \(ITU-T G.711 标准\)](#)

[4 3.3.4 公式验证](#)

[5 3.3.5 最大值 4032 的推导](#)

[6 3.3.6 位提取操作详解](#)

[7 3.3.7 左移运算详解](#)

[8 3.3.8 完整数据流示例 \(以 0xD5 为例\)](#)

[4 3.4 步骤 3: 缩放到模拟电压](#)

[5 3.5 dBm0 换算](#)

- 4. [完整 C 代码（正确实现）](#)
- 5. [常用 PCM 码速查表](#)
- 6. [现有代码中的错误说明](#)
 - 1. [6.1 A-law 解码公式错误](#)
 - 2. [6.2 LINEAR_MAX 值不匹配](#)
 - 3. [6.3 符号逻辑反转](#)
- 7. [手册出处汇总](#)
- 8. [适用范围](#)

TP3057 PCM 码转线性电压值 — 完整说明

本文档严格基于 **TP3057 芯片手册**（[TP3057N-NOPB_规格书_WJ485475.PDF](#)）与 **ITU-T G.711 A-law 标准**，说明如何将 TP3057 输出的 PCM 码字转换为对应的模拟电压值。

1. TP3057 PCM 编码格式

1.1 基本参数

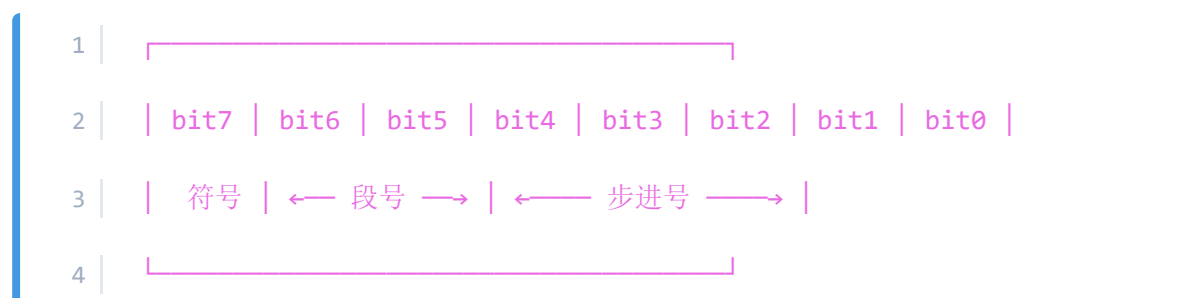
属性	值	手册出处
编码标准	G.711 A-law	p.11 "A-Law"
每采样位数	8 bits	p.4 "The 8-bit code is then loaded..."
附加处理	偶位反转 (Even Bit Inversion)	p.11 "(Includes Even Bit Inversion)"
总码字数	256 (0x00 ~ 0xFF)	8-bit = 256 组合
帧格式	MSB-first, 每帧 8 个 BCLK 数据位 + 248 空闲	p.3 Short/Long Frame Sync

1.2 8-bit A-law PCM 编码位结构详解

为什么需要 A-law 压缩？

人耳对小声敏感、对大声不敏感。如果用 13-bit 线性编码 (-4096 ~ +4095)，小声的量化误差和高声一样大，浪费了大量码字在大信号上。A-law 对数压缩用 8 位表示 13 位的动态范围，把更多码字分配给小声区，用更少码字覆盖高声区。

8-bit 结构: 1 + 3 + 4



bit7 — 符号位 (Sign)

- ▶ 1 → 正电压 (输出线性值为正)
- ▶ 0 → 负电压 (输出线性值为负)

这是线性 PCM 世界中 +/- 的分界线。A-law 是对称的，正负各用相同编码规则，只有这个 bit7 不同。

bit6-bit4 — 段号 (Segment, 0-7)

A-law 把信号幅度分成 8 段，每段的量化步长不同：

段号	覆盖的线性范围	步长	段内码字数
0	0 ~ 31	2	16
1	32 ~ 63	2	16
2	64 ~ 127	4	16
3	128 ~ 255	8	16
4	256 ~ 511	16	16
5	512 ~ 1023	32	16
6	1024 ~ 2047	64	16
7	2048 ~ 4032	128	16

关键规律：**段号每 +1，步长翻倍**。从段 1 到段 7，步长按 2→4→8→16→32→64→128 递增。段 0 是特殊的——它是折线逼近 A-law 在 origin 附近的线性段。

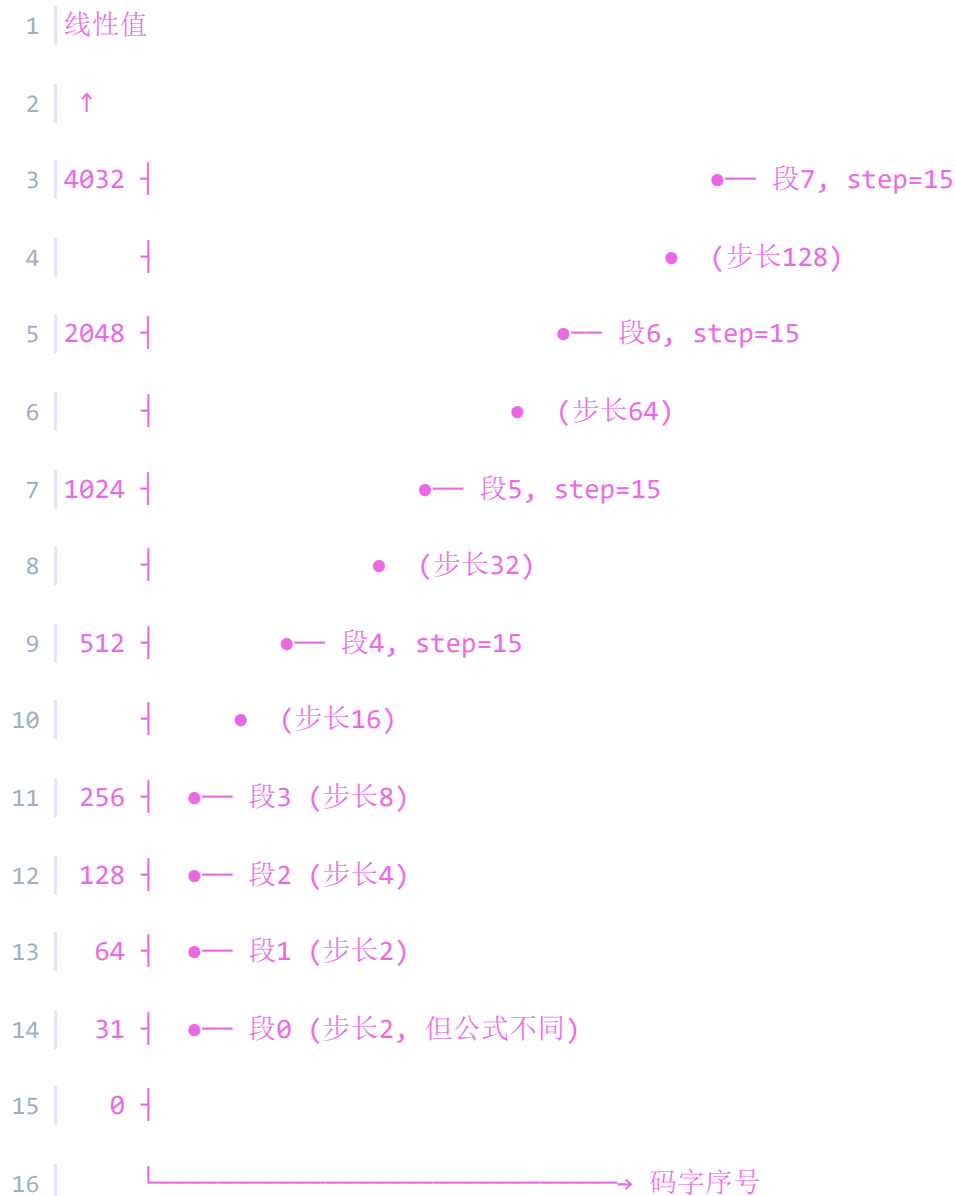
“

解码公式里 $\ll (\text{segment} - 1)$ 的作用就是“翻倍 segment-1 次”。段 2 步长是段 1 的 2 倍，段 3 是段 1 的 4 倍……

bit3–bit0 — 步进号 (Step, 0–15)

在给定的一段内，有 16 个均匀分布的码字 (step 0 ~ 15)。段内步长恒定，step 每 +1，线性值增加一个段步长。

段号控制步长的直观图示



- 18 | 段内每个 step 跨越的距离:
- 19 | 段0: 2 (均匀, 16 个码字覆盖 1~31)
- 20 | 段1: 2 (覆盖 33~63, 步长开始翻倍)
- 21 | 段2: 4
- 22 | 段3: 8
- 23 | 段4: 16
- 24 | 段5: 32
- 25 | 段6: 64
- 26 | 段7: 128 (每 step 跳 128 个线性单位!)

段 7 里 step 从 14 变到 15, 线性值跳 128; 而在段 0, step 变 1 只跳 2。这就是**对数压缩的核心**: 大声信号的量化粗糙 (大步长), 小声信号的量化精细 (小步长)。

解码过程完整示例 (以 0xAA = +Full Scale 为例)

```
1 | TP3057 原始码: 0xAA = 1010 1010
2 | XOR 0x55:      1111 1111 (恢复标准 A-law)
3 |                | |   |
4 |                | |   └─ step  = 1111 = 15
5 |                | └─── segment = 111 = 7
6 |                └─── sign = 1 (正)
7 |
8 | segment = 7, step = 15
9 | → linear = (2×15 + 33) << (7-1)
10 |           = (30 + 33) << 6
11 |           = 63 × 64
12 |           = 4032           ← A-law 正满量程!
```

再看 0xD5 (零电压) :

```
1 | TP3057 原始码: 0xD5 = 1101 0101
2 | XOR 0x55:      1000 0000 (恢复标准 A-law)
3 |                |   |
4 |                |   └─ step  = 0000 = 0
5 |                └─── segment = 000 = 0
6 |                └─── sign = 1 (正)
7 |
8 | segment = 0, step = 0
9 | → linear = 2×0 + 1 = 1     ← 接近零的正最小值
```

负向: 0x2A → XOR 0x55 → 0111 1111 → sign=0, segment=7, step=15 → linear = -4032

代码中 bit 提取的完整流程 (以求 0xAA 为例)

```

1 | alaw_std = tp3057_byte ^ 0x55;          // 0xAA ^ 0x55 = 0xFF = 1111
   | 1111

2 |

3 | sign      = alaw_std & 0x80;          // 1111 1111 & 1000 0000 = 1000
   | 0000 ≠ 0 → 正

4 | magnitude = alaw_std & 0x7F;          // 1111 1111 & 0111 1111 = 0111
   | 1111

5 | segment    = (magnitude >> 4) & 0x07; // 0111 1111 >> 4 = 0000 0111 →
   | 7

6 | step       = magnitude & 0x0F;          // 0111 1111 & 0000 1111 = 0000
   | 1111 → 15

7 |

8 | linear     = (2*15+33) << (7-1) = 63 << 6 = 4032

```

编码字段总结

字段	位置	含义	对线性值的影响
Sign	bit7	1=正, 0=负	决定最终线性值的正负号
Segment	bit6– bit4	段号 0–7	控制步长大小（每段×2），决定线性值的数量级
Step	bit3– bit0	步进号 0–15	段内微调，每+1线性值增加一个段步长

一句话概括：**符号定方向，段号定规模，步进号定段内偏移。**

1.3 偶位反转

TP3057 在标准 A-law 基础上对每个**偶数位**（bit 0, 2, 4, 6）取反，等效于 **XOR 0x55**（**01010101**）：

```

1 | TP3057_PCM = 标准A-law_PCM XOR 0x55

2 | 标准A-law_PCM = TP3057_PCM XOR 0x55    （可逆）

```

1.4 手册中的编码对照表 (p.11)

VIN (at GSX)	TP3057 输出 (含偶位反转)	标准 A-law	含义
+Full-Scale	10101010 (0xAA)	0xFF	正最大值
0V	11010101 (0xD5)	0x80	正零
-Full-Scale	00101010 (0x2A)	0x7F	负最大值

验证偶位反转：

- ▶ $0xAA \wedge 0x55 = 0xFF \checkmark$ (A-law +Full)
- ▶ $0xD5 \wedge 0x55 = 0x80 \checkmark$ (A-law 正零)
- ▶ $0x2A \wedge 0x55 = 0x7F \checkmark$ (A-law -Full)

2. 关键电压参数 (手册 p.9 Transmission Characteristics)

参数	数值	说明
0 dBm0 参考电平	1.2276 Vrms (600Ω 负载)	名义增益 0 dB 的参考输入/输出
tMAX — Max Overload Level	3.14 dBm0 / 2.492 VPK	TP3057 A-law 最大过载电平 (CCITT G.711 "Virtual Decision Value")，对应 PCM ±Full-Scale
(对比) TP3054 tMAX	3.17 dBm0 / 2.501 VPK	TP3054 μ-law 的过载点 (同一行标注值，非 TP3057 参数)

“注：手册第 10 页 Transmission Characteristics 表中 tMAX 一行同时标注了 TP3054 (3.17 dBm0 / 2.501 VPK) 和 TP3057 (3.14 dBm0 / 2.492 VPK)。
TP3057 的 tMAX 明确为 2.492 VPK，这就是满量程参考电压 VPEAK_FS 的手册出处。2.501 VPK 属于 TP3054 (μ-law)，不可混淆。”

换算关系：


```
1 | 0 dBm0 = 1.2276 Vrms = 1.2276 × √2 = 1.736 VPK (正弦波)
2 |
3 | 3.14 dBm0: 电压比 = 10^(3.14/20) = 1.4356
4 | VPK = 1.736 × 1.4356 = 2.492 VPK ✓
```

3. 转换流程

3.1 三步法总览

```
1 | TP3057 PCM 字节 (含偶位反转)
2 |
3 | ▼
4 | [步骤1] XOR 0x55 → 标准 G.711 A-law 字节
5 |
6 | ▼
7 | [步骤2] G.711 A-law 解码 → 13-bit 线性值 (±4032)
8 |
9 | ▼
10 | [步骤3] 缩放: linear × (2.492 / 4032) → 峰值电压 VPK
11 |
12 | ▼
13 | [可选] VPK / √2 → 有效值电压 Vrms (仅正弦波)
```

3.2 步骤 1: 去除偶位反转

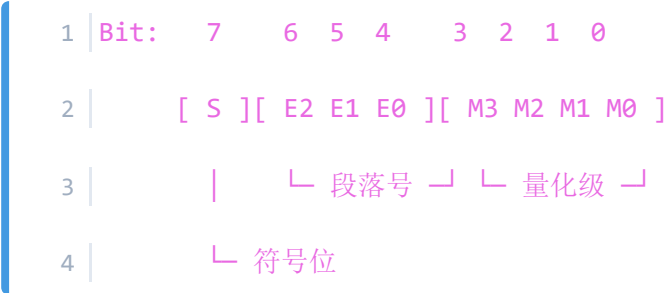
```
1 | unsigned char standard_alaw = tp3057_byte ^ 0x55;
```

0x55 = 01010101 (bit 0, 2, 4, 6 为 1) , 与偶位反转正好抵消。

3.3 步骤 2: G.711 A-law 解码 → 13-bit 线性值

3.3.1 A-law 字节结构

标准 A-law 字节（去除偶位反转后）的结构：



字段	位置	含义
S (sign)	bit 7	符号：1 = 正, 0 = 负
E [2:0] (exponent/segment)	bits 6-4	段落号, 0-7 (越大离零越远)
M [3:0] (mantissa/step)	bits 3-0	段内量化级, 0-15

3.3.2 A-law 段落结构

A-law 将对数压缩的模拟信号分为 8 段，每段 16 级：

段落 s	量化步长	线性值范围 (解码后)
0	2	1 ~ 31
1	2	33 ~ 63
2	4	66 ~ 126
3	8	132 ~ 252
4	16	264 ~ 504
5	32	528 ~ 1008
6	64	1056 ~ 2016
7	128	2112 ~ 4032

3.3.3 解码公式 (ITU-T G.711 标准)

```

1 | int segment = (standard_alaw >> 4) & 0x07; // 段落 0-7
2 | int step    = standard_alaw & 0x0F;        // 量化级 0-15
3 | int sign    = (standard_alaw & 0x80) ? 1 : -1; // 1=正, -1=负
4 |
5 | int linear_13bit;
6 |
7 | if (segment == 0) {
8 |     linear_13bit = 2 * step + 1;              // 范围: 1 ~ 31
9 | } else {
10 |     linear_13bit = (2 * step + 33) << (segment - 1); // 范围: 33 ~
    4032
11 | }
12 |
13 | linear_13bit = sign * linear_13bit; // 加上符号, 范围: -4032 ~ +4032

```

3.3.4 公式验证

标准 A-law	segment	step	公式计算	结果
0xFF (+Full)	7	15	$(30+33) \ll 6 = 63 \times 64$	+4032 ✓
0x80 (正零)	0	0	$2 \times 0 + 1$	+1 ✓
0x7F (-Full)	7	15	$-(30+33) \ll 6$	-4032 ✓
0x00 (负零)	0	0	$-(2 \times 0 + 1)$	-1 ✓

“**说明：**手册指出 TP3057 仅输出“正零”（p.11 “PCM Code Equals Positive Zero”），正常工作时不会出现 0x00（负零）。

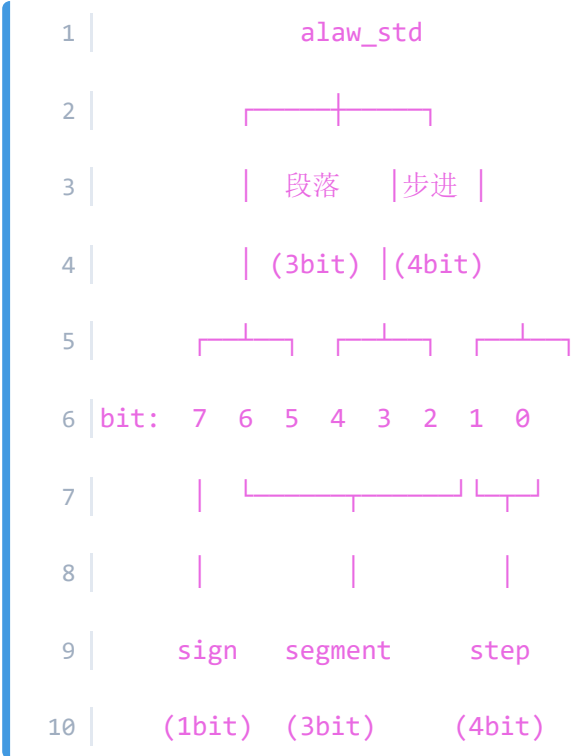
3.3.5 最大值 4032 的推导

```
1 | 取 s = 7（最大段落），m = 15（最大量化级）：  
2 |  
3 | linear_max = (2 × 15 + 33) × 2^(7-1)  
4 |           = (30 + 33) × 2^6  
5 |           = 63 × 64  
6 |           = 4032
```

这是 **G.711 A-law 标准的数学定义**，不是经验值或测量值。

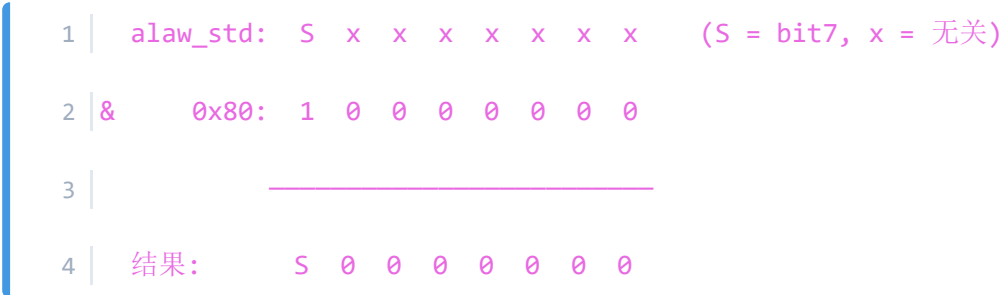
3.3.6 位提取操作详解

解码之前，需要先用位操作将 A-law 字节**解剖**为三个独立字段。这是 `pcm_to_voltage_rms` 中提取 `sign`、`segment`、`step` 的 4 行代码：



① `sign = alaw_std & 0x80;` — 提取符号位 (bit 7)

`0x80` = `1000 0000`, 只有 bit 7 是 1。按位与 `&` 的规则是两个 bit 都为 1 结果才为 1:



0 与任何 bit 做 `&` 都得 0, 所以 bit 6~0 全部清零, 只有 bit 7 原样保留。

alaw_std	<code>& 0x80</code> 结果	含义
<code>0xFF</code> (+Full)	<code>0x80</code> (非零)	bit 7 = 1 → 正电压
<code>0x80</code> (正零)	<code>0x80</code> (非零)	bit 7 = 1 → 正电压
<code>0x7F</code> (-Full)	<code>0x00</code> (零)	bit 7 = 0 → 负电压

后续用 `if (!sign)` 判断: `sign == 0` 时取反 (负), 否则保持正。

② `magnitude = alaw_std & 0x7F;` — 屏蔽符号位, 保留幅度

`0x7F` = `0111 1111`, bit 7 是 0、bit 6~0 全是 1:

```

1 | alaw_std:  S  E2 E1 E0 M3 M2 M1 M0
2 | & 0x7F:  0  1  1  1  1  1  1  1
3 |
4 | magnitude:  0  E2 E1 E0 M3 M2 M1 M0

```

bit 7 被清零 (无论正负) , bit 6~0 (段落号 + 量化级) 原样保留。magnitude 编码的是信号离零点的**绝对距离**。

③ `segment = (magnitude >> 4) & 0x07;` — 提取段落号 (bit 6~4)

分两步：先右移对齐，再截断取低 3 位。

>> 4 将 bit 6~4 移到 bit 2~0 位置：

```

1 | magnitude:  0  E2 E1 E0 M3 M2 M1 M0
2 | >> 4 后:    0  0  0  0  0  E2 E1 E0  (段落号被移到了低 3 位)

```

`& 0x07` = `& 0000 0111`，只保留低 3 位，确保不会带入多余数据：

```

1 | >>4 后:    0  0  0  0  x  E2 E1 E0
2 | & 0x07:    0  0  0  0  0  1  1  1
3 |
4 | segment:    0  0  0  0  0  E2 E1 E0  (干净的 0~7)

```

④ `step = magnitude & 0x0F;` — 提取段内量化级 (bit 3~0)

`0x0F` = `0000 1111`，只保留低 4 位：

```

1 | magnitude:  0  E2 E1 E0 M3 M2 M1 M0
2 | & 0x0F:    0  0  0  0  1  1  1  1
3 |
4 | step:       0  0  0  0  M3 M2 M1 M0  (干净的 0~15)

```

完整拆分示例 (0xD5 → 标准 A-law 0x80)

```

1 | 0x80 = 1 0 0 0 0 0 0 0
2 |
3 |           |  ⊣ 000  ⊣ ⊣ 0000  ⊣
4 |           |           |           ⊣ step = 0
5 |           |           ⊣ segment = 0
6 |           ⊣ sign = 1 (正, 0x80 ≠ 0)
7 |
8 | 解码: linear = 2×0 + 1 = 1 ≈ 0V ✓

```

五种位操作总结

操作	语法	效果
按位与 &	<code>x & 0x80</code>	只保留 bit 7，其余清零—— 提取
按位与 &	<code>x & 0x7F</code>	只清零 bit 7，其余保留—— 屏蔽
右移 >>	<code>x >> 4</code>	将高位移到低位—— 对齐
按位与 &	<code>x & 0x07</code>	只保留低 3 位—— 截断
按位与 &	<code>x & 0x0F</code>	只保留低 4 位—— 截断

3.3.7 左移运算详解

解码公式中对 $\text{segment} > 0$ 的情况使用了左移:

```
1 | linear = (2 * step + 33) << (segment - 1);
2 | //      ┌──────────┐      ┌──────────┐
3 | //      │           │      │           │
           基础值          指数放大
```

<< 左移运算符

`x << n` 将 `x` 的二进制位全部向左移动 `n` 位，右侧空位补 0。效果等同于 `x × 2n`：

1	$1 \ll 0 = 1$	(×1)
2	$1 \ll 1 = 2$	(×2)
3	$1 \ll 2 = 4$	(×4)
4	$1 \ll 3 = 8$	(×8)
5	$1 \ll 6 = 64$	(×64)

在此公式中的作用

(segment - 1) 控制左移位数，让量化步长随段落号翻倍——这正是 G.711 对数压缩的核⼼机制：

segment	segment - 1	<< 效果	等效乘法	量化步长
0	— (走特例分支)	—	—	2
1	0	不移	×1	2
2	1	左移 1 位	×2	4
3	2	左移 2 位	×4	8
4	3	左移 3 位	×8	16
5	4	左移 4 位	×16	32
6	5	左移 5 位	×32	64
7	6	左移 6 位	×64	128

同一量化级在不同段落的演算

以 step = 5 为例，观察段落号增加时结果如何指数增长：


```

1 | segment=1: (2×5+33) << 0 = 43 × 1 = 43
2 | segment=2: (2×5+33) << 1 = 43 × 2 = 86
3 | segment=3: (2×5+33) << 2 = 43 × 4 = 172
4 | segment=4: (2×5+33) << 3 = 43 × 8 = 344
5 | segment=5: (2×5+33) << 4 = 43 × 16 = 688
6 | segment=6: (2×5+33) << 5 = 43 × 32 = 1376
7 | segment=7: (2×5+33) << 6 = 43 × 64 = 2752

```

为什么用左移而不是直接写乘法

三种写法完全等价：

```

1 | linear = (2 * step + 33) << (segment - 1);           // 二进制移位
2 | linear = (2 * step + 33) * (int)pow(2.0, segment - 1); // 数学写法
3 | linear = (2 * step + 33) * (1 << (segment - 1));     // 合并写法

```

⏪ 写法最直观：段落号每增加 1、步长加倍——3 个 bit 的段落号覆盖 128 倍动态范围，这正是 A-law 对数压缩的精髓。

3.3.8 完整数据流示例（以 0xD5 为例）

将从 TP3057 输出到最终电压的完整转换过程串联起来：

```

1 | TP3057 输出:  0xD5
2 |
3 | [步骤1] XOR 0x55:  0xD5 ^ 0x55 = 0x80
4 |
5 | 标准 A-law:      0x80 = 1 0 0 0  0 0 0 0
6 |
7 |               |  |  |  |  |  |  |  |
8 |               |  |  |  |  |  |  |  |
9 |               |  |  |  |  |  |  |  |
10 |              |  |  |  |  |  |  |  |
11 | [步骤2] 解码:  segment==0 → 2×0 + 1 = 1
12 |
13 | [步骤3] 符号:  sign≠0 → 不取反, linear = +1
14 |
15 | [步骤4] 缩放:  1 × (2.492/4032) = 0.000618 VPK
16 |
17 | [步骤5] RMS:   0.000618 / 1.414 = 0.000437 Vrms
18 |
19 |

```

▼

≈ 0V 的模拟输入 ✓

3.4 步骤 3: 缩放到模拟电压

从手册参数: ±Full-Scale PCM 码 (linear = ±4032) 对应 GSX/VFRO 引脚 ±**2.492** VPK。

```

1 | V_GSX_PK = linear_13bit × (2.492 / 4032)
2 |
3 |           = linear_13bit × 0.00061806    V/unit
4 |
5 | V_GSX_RMS = V_GSX_PK / √2                (仅适用于正弦波信号)
6 |
7 |           = |V_GSX_PK| / 1.41421356

```

3.5 dBm0 换算

1 | $\text{dBm0} = 20 \times \log_{10}(|\text{linear_13bit}| / 4032) + 3.14$

2 |

3 | 验证: $\text{linear} = 4032 \rightarrow \text{dBm0} = 20 \times \log_{10}(1) + 3.14 = +3.14 \text{ dBm0} \checkmark$

4 | $\text{linear} = 2808 \rightarrow \text{dBm0} = 20 \times \log_{10}(0.6964) + 3.14 \approx -3.14 + 3.14 = 0 \text{ dBm0} \checkmark$

5 | (0 dBm0 对应 1.2276 Vrms, linear 值为 $4032 / 10^{(3.14/20)} \approx 2808$)

4. 完整 C 代码（正确实现）

```
1  /*
2  * TP3057 PCM 字节 → GSX/VFRO 峰值电压
3  * 严格基于 TP3057 手册 + ITU-T G.711 A-law 标准
4  *
5  * 参数: tp3057_byte – TP3057 输出的 PCM 码字（含偶位反转）
6  * 返回: 对应的模拟峰值电压 VPK（正负取决于信号极性）
7  */
8
9  #define VPEAK_FS      2.492          /* TP3057 A-law ±Full-Scale 峰值电压
   (V) – 手册 p.9 */
10 #define LINEAR_MAX    4032          /* G.711 A-law 13-bit 线性最大值 –
   ITU-T G.711 */
11
12 double tp3057_pcm_to_voltage_pk(unsigned char tp3057_byte)
13 {
14     int segment, step, sign;
15     int linear_13bit;
16     double vpeak;
17
18     /* 步骤 1: 去除偶位反转, 恢复标准 A-law */
19     unsigned char standard_alaw = tp3057_byte ^ 0x55;
20
21     /* 步骤 2: 标准 G.711 A-law 解码 → 13-bit 线性值 (±4032) */
22     sign      = (standard_alaw & 0x80) ? 1 : -1;      /* 1=正, -1=负 */
23     segment   = (standard_alaw >> 4) & 0x07;          /* 段落号 0-7 */
24     step      = standard_alaw & 0x0F;                 /* 量化级 0-15 */
25
```

```

26 |     if (segment == 0) {
27 |         linear_13bit = 2 * step + 1;                                /* 范围：1 ~ 31
    |     */
28 |     } else {
29 |         linear_13bit = (2 * step + 33) << (segment - 1); /* 范围：33 ~
    |         4032 */
30 |     }
31 |     linear_13bit = sign * linear_13bit;                            /* 范围：-4032 ~
    |     +4032 */
32 |
33 |     /* 步骤 3：缩放到峰值电压 */
34 |     vpeak = (double)linear_13bit * (VPEAK_FS / (double)LINEAR_MAX);
35 |
36 |     return vpeak; /* 单位：VPK */
37 | }
38 |
39 | /*
40 |  * 峰值 → 有效值（仅正弦波信号）
41 |  */
42 | double tp3057_pcm_to_voltage_rms(unsigned char tp3057_byte)
43 | {
44 |     double vpeak = tp3057_pcm_to_voltage_pk(tp3057_byte);
45 |     return vpeak / 1.4142135623730951; /* vpeak / sqrt(2) */
46 | }

```

5. 常用 PCM 码速查表

TP3057 输出	标准 A-law	linear_1 3bit	V_GSX (VPK)	V_GSX (Vrms)	dBm 0	含义
0xAA	0xFF	+4032	+2.492	+1.762	+3.14	+Full Scale
0xD5	0x80	+1	+0.0006	+0.0004	-69.1	正零 (≈0V)
0x2A	0x7F	-4032	-2.492	1.762	+3.14	-Full Scale
0x55	0x00	-1	-0.0006	0.0004	-69.1	负零 (不应出现)

6. 现有代码中的错误说明

项目现有代码（`shared.cpp` 中的 `pcm_to_voltage_rms`）存在以下问题：

6.1 A-law 解码公式错误

现有代码：

```
1 | if (segment == 0) {
2 |     linear = (step << 4) + 8;           // = 16×step + 8
3 | } else {
4 |     linear = ((0x10 | step) << (segment + 2)) + 8; // =
      (16+step)×2^(seg+2) + 8
5 | }
```

此公式不是标准 G.711 解码。最大值输出约 15880（而非 4032），且段间出现逆序（segment 0 最大值 248 > segment 1 最小值 136）。

正确公式：见本文档第 3.3.3 节。

6.2 LINEAR_MAX 值不匹配

现有代码：`#define LINEAR_MAX 8031`

8031 既不符合标准 A-law 最大值 4032，也不匹配现有解码公式的 ~15880。导致电压缩放偏差约 2-4 倍。

正确值：`#define LINEAR_MAX 4032`（配合标准 A-law 解码公式）。

6.3 符号逻辑反转

现有代码：

```
1 | if (sign) linear = -linear;    // MSB=1（正）时取反！错误！
```

手册明确规定 MSB=1 对应正电压（p.11 编码表：0xFF = +Full-Scale）。现有代码在 MSB=1 时执行取反，导致正负完全颠倒，`log10(负数)` 产生 NaN。

正确写法：

```
1 | if (!sign) linear = -linear;    // MSB=0（负）时取反
2 | // 或先提取符号再统一处理：linear_13bit = sign * linear;
```

7. 手册出处汇总

关键信息	手册位置
TP3057 使用 A-law 编码	p.1 "A-law 16-pin TP3057"
8-bit PCM	p.4 "The 8-bit code is then loaded..."
偶位反转	p.11 " (Includes Even Bit Inversion) "
编码格式表 (+Full / 0V / -Full)	p.11 "ENCODING FORMAT AT DX OUTPUT"
0 dBm0 = 1.2276 Vrms	p.9 "0 dBm0 = 1.2276 Vrms"
TP3057 过载点 3.14 dBm0, 2.492 VPK	p.9 "GXA ... TP3057 (3.14 dBm0) ... 2.492 VPK"
正零码	p.11 "PCM Code Equals Positive Zero"
tMAX = 2.501 VPK	p.9 "tMAX — Max Overload Level"
接收通道单位增益	p.4 "The receive section is unity-gain"
VFRO 输出驱动 ±2.5V (600Ω)	p.9 "VRO — Receive Output Drive Level"

8. 适用范围

此转换公式 **对称适用于收发两个方向**：

方向	PCM 位置	模拟电压位置	增益
发送 (Encode)	DX 引脚输出	GSX 引脚 (运放输出端)	由外部电阻 R1/R2 设定
接收 (Decode)	DR 引脚输入	VFRO 引脚输出	单位增益 (可加 T-pad 衰减)

手册 p.4: "The receive section is unity-gain."——接收通道为单位增益，0 dBm0 数字码在 VFRO 产生 1.2276 Vrms (600Ω 负载)。

“

发送增益：手册 Figure 4 (p.13) ，通过外接 R1、R2 在 GSX 处设置增益：Gain = $20 \cdot \log_{10}((R1+R2)/R2)$ ， $R1+R2 \geq 10k\Omega$ 。0 dBm0 的参考输入在 VFX1a 端测量，经增益放大后在 GSX 端达到 1.2276 Vrms (单位增益时) 。